



Unit II: Hashing and Collisions

Collision Resolution in Hashing

PPT PRESENTED BY

NAME : Srinivas Rao Tammireddy

ROLL NO : 257Y1D5805

YEAR : I Year I Semester

ACADEMIC YEAR : 2025 - 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Hashing and the Collision Problem

Hashing maps keys of arbitrary size to fixed-size values (hash codes) using a hash function. A collision occurs when two distinct keys hash to the same table slot.

Core Concepts

- **Hash Table:** An array of size m that stores key-value pairs.
- **Hash Function $h(k)$:** Maps key k to an index in $[0, m-1]$. Ideal function is uniform (distributes keys equally).
- **Load Factor (α):** $\alpha = n / m$, where n is the number of keys and m is the table size. Indicates how full the table is.
- **Collision Definition:** If $k_1 \neq k_2$ but $h(k_1) = h(k_2)$, they contend for the same array index.

Why Collisions are Unavoidable

- **Pigeonhole Principle:** If the universe of keys U is larger than table size m , multiple keys **MUST** map to the same slot.
- **Birthday Paradox:** In a table of size 365, there is a 50% chance of collision with only 23 keys inserted, even with a perfect random hash function.
- **Resolution Requirement:** Hashing requires an efficient collision resolution strategy to maintain $O(1)$ average lookup speed.

Separate Chaining (Open Hashing)

Separate Chaining resolves collisions by maintaining a linked list of elements at each slot:

How it Works

- **Slots contain pointers:** Each array entry points to the head of a linked list (or other dynamic container).
- **Insert:** Hash key to slot i . Insert new element at the head of list i . $O(1)$ time.
- **Search:** Hash key to slot i . Traverse list i to find key. $O(1 + \alpha)$ average time.
- **Delete:** Hash key to slot i . Find key in list i and remove the node. $O(1 + \alpha)$ average time.

Pros & Cons

- ✓ **Simple to implement.**
- ✓ **Table never fills up:** Load factor α can exceed 1 without catastrophic failure.
- ✓ **Graceful degradation:** Performance degrades linearly as α grows.
- ✗ **Memory overhead:** Extra memory is wasted storing pointers for linked list nodes.
- ✗ **Cache inefficiency:** Linked list nodes are scattered in memory, causing CPU cache misses.

Open Addressing (Closed Hashing)

Open Addressing stores all elements directly within the array. It probes subsequent slots on collision:

1. Linear Probing

Checks consecutive slots: $h(k, i) = (h'(k) + i) \% m$.

- Easy to implement.
- High cache locality.
- Primary Clustering: Long runs of occupied slots build up, slowing down searches.

2. Quadratic Probing

Checks slots quadratically: $h(k, i) = (h'(k) + c1 \cdot i + c2 \cdot i^2) \% m$.

- Reduces primary clustering.
- Secondary Clustering: Keys that hash to the same index traverse the same probe path.

3. Double Hashing

Uses a second hash function to determine step size: $h(k, i) = (h1(k) + i \cdot h2(k)) \% m$.

- Best distribution: step size depends on key.
- Eliminates clustering.
- $h2(k)$ must be coprime to m .



Comparison: Chaining vs. Open Addressing

A direct structural and performance comparison between the two strategies:

Feature	Separate Chaining	Open Addressing
Storage Location	Outside array (pointers/lists)	Inside the primary array slots
Max Load Factor (α)	α can be > 1 (limitless)	α strictly ≤ 1 (typically keeps $\alpha < 0.7$)
Cache Performance	Poor (linked list traversals)	Excellent (array elements adjacent)
Deletion Complexity	Simple (standard node deletion)	Complex (requires 'deleted' placeholders)
Memory Overhead	Extra memory spent on pointers	Wasted array space if table is sparse

Applications of Hash Tables

Hashing and collision resolution are critical parts of high-performance software engineering:

Symbol Tables in Compilers

Compilers and interpreters use hash tables to store identifier names (variables, functions) and their metadata. High lookup speeds are vital during syntactic analysis and code generation.

Database Query Caching

Database engines cache parsed queries and execution plans in hash tables for instant retrieval, avoiding costly re-compilation of query strings.

IP Routing Table Lookups

Routers use hash tables to store IP routing routes, enabling quick route matching to direct network packets with minimum latency.

Set Implementations & deduplication

Language runtimes implement 'Set' classes (e.g., HashSet in Java/C#) using hash tables to support $O(1)$ membership checks and fast duplicate elimination.



THANK YOU

Department of Computer Science and Engineering
Marri Laxman Reddy Institute of Technology and Management